Sanad | Alejandro | Alicia | Rodrigo | Khadeja

FireFront-HPC

A Parallel Wildfire Spread Simulator

1. The idea

We will write a wildfire spread simulator ourselves and use it to study how MPI, OpenMP and GPU programming change performance, scalability and resource use. The landscape is a regular 2-D grid. Each cell is **unburnt**, **burning** or **burnt**. At every time step an unburnt cell can be ignited by any of its eight burning neighbours; a burning cell stays alight for *B* steps and then becomes burnt.

The topic is current (2025 was one of Spain's worst fire seasons) and it is computationally demanding in the way that justifies HPC: going from 100 m to 25 m resolution multiplies the cell count by 16, and a useful forecast needs hundreds of weather scenarios rather than one.

Scope. The **minimum deliverable** is serial → MPI → hybrid MPI+OpenMP → one measured optimization, on a synthetic landscape. On its own this covers every required experiment. The GPU port is kept small on purpose so that it cannot delay the report, and real terrain data is a stretch goal.

Tier	Content	Covers
Core (must)	Serial baseline (already written); MPI with strips and blocks; hybrid MPI+OpenMP; overlap optimization; profiling. Synthetic, seeded landscape.	Baseline, 2 HPC approaches, scaling, sensitivity, optimization, bottleneck analysis
Extra technology (must, small)	Update kernel ported to CUDA, with an OpenACC version to compare; 1 GPU, one grid size. *Fallback* if the GPU queue is blocked: OpenACC built for multicore CPU and compared with OpenMP.	"Additional HPC technology"; a negative result is acceptable
Stretch	Tuned GPU version (resident data, several scenarios per job); real DEM and land cover (IGN, Copernicus); HDF5 output.	Better report and a stronger EuroHPC case

Success metric. The project succeeds if (1) every parallel version reproduces the serial result bit for bit, (2) experiments E1–E8 are complete and reproducible from `REPRODUCE.md`, and (3) the main bottleneck is explained with counter or timer evidence, not just stated. Performance target (a goal, not a condition): ≥ 60% strong-scaling efficiency at 64 cores on the L grid (sizes defined in §5).

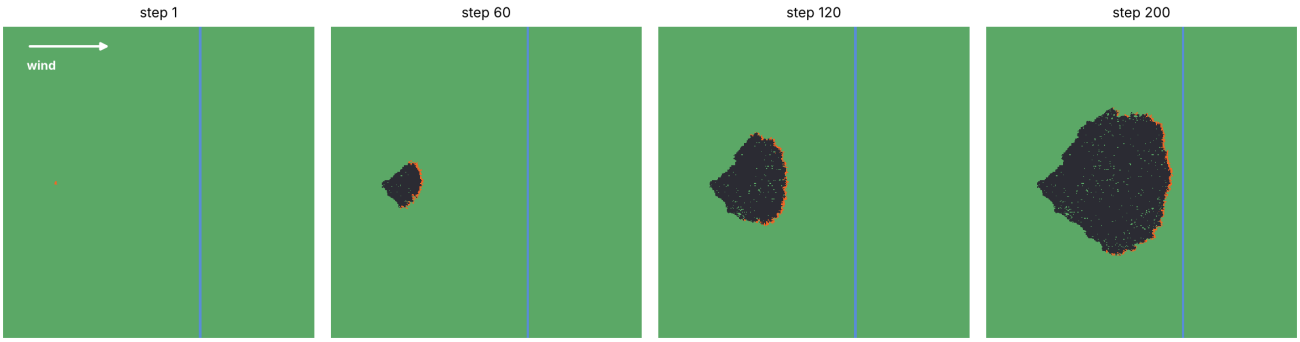


Figure 1 – Output of our serial C code (400×400 cells, default parameters, wind towards the east). The fire spreads mainly downwind and stops at the river (blue, fuel = 0). With a single ignition point most of the grid never burns, which is why timing runs will use 16 ignition points (§2).

2. Model details

This section describes the model **as implemented** in the repository (`src/`). Where the code still has to change, it is marked *planned*.

Item	Specification
Grid and state	2-D raster, N×N cells with a one-cell border (`stride = nx + 2`), so the same loop works for an MPI rank with neighbours. Per cell: one **state byte** that also holds the burn age (0 = unburnt, 1…B = burning for that many steps, 255 = burnt), fuel (float) and elevation (float). The state grid is double-buffered; fuel and elevation are read-only. About 10 bytes/cell, so the 8192² grid needs ≈ 0.7 GB.

Item	Specification
Time step	One step is one unit of spread time. In the timing experiments we fix the **number of steps**, so runs at different resolutions compare work per step. For a fixed *simulated* time, going from 100 m to 25 m cells costs ≈ 64× more work (16× more cells, ≈ 4× more steps); that number is used in the scale-up plan (§7).
Neighbourhood	Moore (8 neighbours), so the halo is one cell wide. With 2-D blocks a rank also needs its four **diagonal** neighbours' corner cells (handled in §3).
Update scheme	**Dense:** every cell is visited every step, even far from the fire, so the work per rank is nearly uniform (cells next to a burning neighbour do a little more work). This keeps the scaling experiments clean. Updating only the active region is a possible later optimization; `tools/prototype/imbalance_demo.py` already estimates the load imbalance it would cause.
Ignition probability	For each burning neighbour d: $\text{prob}_d = p_0(1 - H) \cdot \text{fuel} \cdot f_{\text{wind},d} / \text{dist}_d \cdot f_{\text{slope}}$, clamped to 1, and $p = 1 - \prod_d(1 - \text{prob}_d)$. Wind: $f_{\text{wind}} = \exp(c_w \cdot V \cdot (\cos\theta - 1))$, which is 1 straight downwind and smaller otherwise. Slope: $f_{\text{slope}} = \exp(c_s \cdot \Delta h / (\text{dist} \cdot \text{cell size}))$, with Δh rounded to whole metres (±60 m). Both are **pre-computed tables** (`src/rules.h`), so there is no exp() call inside the cell loop.
Parameters (defaults)	$p_0 = 0.8$; B = 3 steps; cell size 25 m; wind V = 6 m/s towards the east; $c_w = 0.25$ s/m; $c_s = 1.5$; humidity H = 0.2; seed 42; a 3-cell river at 63% of the width. Scenarios for the sensitivity runs: H = {0.2, 0.4, 0.6}, V = {0, 6, 12} m/s. All values can be changed on the command line (`--help`).
Ignition	Current code: one 3×3 patch at (N/6, N/2). *Planned* (task A2): an `--ignitions N` option with N seeded points (16 in timing runs) so the fire reaches every subdomain. The default stays 1, so existing checksums remain valid.
Randomness	SplitMix64 hash of (seed, global cell index, step), 24 random bits per draw. The result therefore does not depend on the number of ranks or threads.
Landscape	Fuel (0.35–1.0) and elevation (0–120 m) are smooth value noise generated in C from the **global** cell position and the seed, so each rank can build its own part without reading a file.
Outputs	One `RESULT key=value` line per run (variant, grid, steps, seed, ranks, threads, t_total, cells_per_s, burning, burnt, checksum, commit, compiler); the first 10 steps are an untimed warm-up. Optional PGM snapshot of the final state. Timed runs use no snapshot and no progress output. *Planned* for MPI: per-rank halo and reduction times, burnt area reduced every K = 50 steps.
What “validated” means	**Correctness:** every parallel version gives the same checksum as the serial one for the same options before it is timed. **Plausibility checks** (already in `tests/test_model.c`): same seed gives the same result, burnt cells never decrease, a fire in damp conditions goes out, wind pushes the fire downwind, the fire does not cross the river. This is not physical validation; comparison with real fire perimeters is out of scope.

3. HPC design

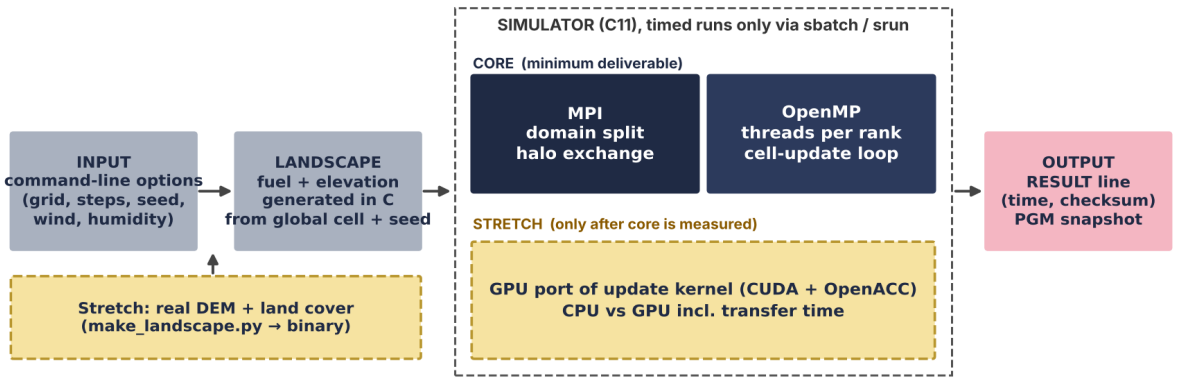


Figure 2 – Software structure. Navy = minimum deliverable, dashed yellow = stretch. Timed runs only go through sbatch/srun on compute nodes.

MPI (main, to do). The grid is split over a Cartesian communicator (`MPI_Cart_create`). Baseline halo exchange uses blocking `MPI_Sendrecv`. We compare two decompositions, which is our main sensitivity study (Figure 3). **1-D strips** need two messages per step, each a full grid row; corner cells are included automatically. **2-D blocks** send ≈ N/√P cells per edge but need the diagonal corners. We use a **two-phase exchange**: first north/south rows, then east/west columns that are one cell longer at each end, so the corners received in phase 1 are passed on (columns sent with `MPI_Type_vector`). The burnt area is accumulated locally and combined with `MPI_Allreduce` only every K = 50 steps and at the end; a per-step reduction would add a synchronisation point every step and cancel the overlap optimization. The Slurm template `slurm/hybrid.sbatch` already expects `--decomp strips|blocks` and `--overlap 0|1` options.

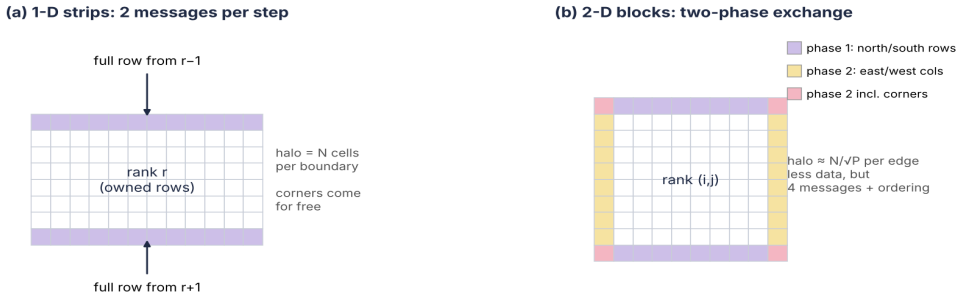


Figure 3 – Halo exchange. Strips: 2 large messages. Blocks: smaller messages but 4 of them, in two ordered phases so that corner cells arrive.

OpenMP (main, code ready). The cell loop already has `#pragma omp parallel for schedule(static)` over rows, and the landscape is initialised in a parallel loop with the same schedule, so memory pages are first touched by the thread that later uses them (first touch). The state grids come from `calloc`, so their pages are only placed when the parallel loop first writes them. Locally the OpenMP build gives the same checksum as serial. Still to do: run it on the cluster with pinning (`OMP_PROC_BIND=close, OMP_PLACES=cores`) and check scaling across both sockets. For the same core count we test several ranks × threads splits, on one and two nodes.

Additional technology – GPU. The update kernel is ported to CUDA, with an OpenACC version as a comparison. The grid stays on the GPU between steps and is copied back only at the end. We compare it with the best one-node CPU configuration, with and without CPU ↔ GPU transfer time. If the GPU result is worse, we report it and explain why.

Optimization. Overlap communication and computation: post `MPI_Isend/MPI_Irecv`, update the interior cells while messages are in flight, then update the one-cell boundary ring. Many MPI libraries only move data inside MPI calls, so a naive version may show **no gain**. We therefore also test calling `MPI_Test` between interior row blocks. If overlap still does not help, we show the timer breakdown that explains why (e.g. halo time already small) instead of hiding the result. Secondary candidates: active-region tracking, cache tiling, vectorization of the inner loop.

Floating-point reproducibility. All CPU builds use `-ffp-contract=off`, so serial, OpenMP, MPI and hybrid runs round the same way and must give the **same checksum**. Because wind and slope factors come from tables, the GPU only differs in how it orders and fuses multiplications. We first try `nvcc --fmad=false` for an exact match; otherwise the GPU version is accepted if the burnt area is within 0.5% of the CPU result, and we report the fraction of cells that differ.

4. Tech stack

Layer	Choice	Notes
Simulator language	**C** (C11)	Standard for HPC; works directly with MPI and OpenMP
Compiler / flags	GCC, `-O3 -march=native -std=c11 -ffp-contract=off` (+ `-fopenmp`); mpicc wrapper	Exact versions recorded from the cluster modules
Message passing	**MPI** – OpenMPI (e.g. `OpenMPI/4.1.5-GCC-12.3.0`)	Launched with `srun --cpu-bind=cores`
Shared memory	**OpenMP**	Threads per rank, pinned
GPU (extra)	**CUDA** (nvcc), **OpenACC** (NVHPC) to compare	Module names still to check; GPU partition
Job scheduling	**Slurm**: `sbatch, srun, sacct, squeue, scontrol`	Templates in `slurm/` take parameters as environment variables
Profiling	STREAM, `MPI_Wtime` timers, `perf stat/record` (LIKWID if available), `nvidia-smi` / Nsight Systems	Compute vs communication, cache misses, memory bandwidth, GPU utilization and transfers
Build and tests	Make: `make, make omp, make test` (MPI and GPU targets to add)	`tests/test_model.c` and `tests/run_all.sh`
Pre/post-processing	**Python 3**: NumPy, matplotlib, pandas; `parse_logs.py`, `make_plots.py` (to do)	Turn RESULT lines into one CSV and all figures; never part of timed runs
Version control	GitHub: `sanadhamadd/firefront-hpc`	Branches + pull requests (CONTRIBUTING.md); commit hash in every RESULT line

5. Experiments and measurements

Grid sizes cover the same 205 × 205 km area at three resolutions: **S** = 2048² (100 m, 4.2 M cells), **M** = 4096² (50 m) and **L** = 8192² (25 m, 67 M cells). Each timed run is 400 steps. A first local test of the serial code gave ≈ 7 ns per cell update (1.4×10^8 cells/s on a 2048² grid), so the serial L run should take ≈ 3–4 min, inside the 10-minute job limit; this is re-checked on the cluster in E1. The cluster is shared, so each configuration is repeated (3× for scaling, 5× for the optimization), one repetition per job, and we report the median with min–max bars. **Assumption to verify (task A1):** core counts below assume 2 CPU nodes with 32 cores each in 2 sockets, and an NVIDIA GPU with CUDA / NVHPC modules; adjust the matrix if the cluster differs.

#	Experiment	Configurations	Runs	Measured
E1	Baseline	Serial, L grid, 1 core	1 × 3	Wall time, MCUPS (million cell updates/s)
E2	Strong scaling	L grid; OpenMP 1, 2, 4, 8, 16, 32 threads (1 node); then MPI strips 1–32 ranks (1 node) and 64 (2 nodes)	13 × 3	Speed-up, efficiency, halo share
E3	Hybrid split	32 cores: 32×1, 8×4, 2×16; 64 cores: 64×1, 16×4, 8×8, 4×16, 2×32 (ranks × threads)	8 × 3	Time, halo time, memory (sacct)
E4	Weak scaling	1024² cells per core: 1 → 4 → 16 → 64 cores (grids S/4, S, M, L)	4 × 3	Weak efficiency T(1)/T(p)
E5	Sensitivity: decomposition	Strips vs blocks at 16, 32, 64 ranks, L grid	6 × 3	Time, messages, communication share
E6	Sensitivity: resolution	S, M, L at 32 cores (100 / 50 / 25 m)	3 × 3	Time per step, memory, MCUPS
E7	Optimization	Blocking vs overlap (± MPI_Test) at 16, 32, 64 ranks, best decomposition	6 × 5	Before/after time and efficiency
E8	Profiling	perf on 1 rank and on 32 ranks; timers in every run	2 × 1	Cache misses, bandwidth, compute vs halo
E9	Extra technology	CUDA and OpenACC kernels, L grid, 1 GPU vs best 1-node CPU	3 × 3	Time with/without transfers, GPU utilization
E10	STREAM + output cost	STREAM triad on 1 node; L grid with and without the PGM snapshot	3 × 3	Node memory bandwidth; I/O share of runtime

In total about 160 short jobs, nearly all under 5 minutes. Speed-up S(p) = T(1)/T(p); efficiency E(p) = S(p)/p; weak-scaling efficiency $E_w(p) = T(1)/T(p)$ at constant cells per core. Wind and humidity change the output but hardly the runtime (dense update); one run per scenario confirms this.

Roofline estimate. One cell update reads the state byte, fuel and elevation (9 B) and writes one state byte (1 B, plus ≈ 1 B write-allocate), so ≈ 11 bytes come from memory per update; neighbour reads hit cache. The upper bound is therefore MCUPS_{max} ≈ memory bandwidth / 11 B. We measure the node bandwidth with STREAM; at e.g. 100 GB/s this gives ≈ 9 000 MCUPS per node. If measured MCUPS approaches this bound, the kernel is memory-bound; if it stays far below, the bottleneck is elsewhere (hashing for the random numbers, branches, communication).

Hypotheses to test. (1) The kernel is limited by memory bandwidth, not arithmetic, so thread scaling flattens before 32 cores per node (roofline above, perf cache-miss and bandwidth counters). (2) A halo row of the L grid is only 8 KB (one state byte per cell), so communication is latency-bound: strips (2 messages) should beat blocks (4 messages in 2 phases) at our scale, while blocks only pay off on far larger systems. (3) Overlap gains little unless MPI progress is forced.

6. Risks and plan

Risk	Mitigation
GPU partition busy or the toolchain is missing	GPU is outside the core; fallback is OpenACC multicore vs OpenMP
Overlap shows no gain	Test MPI_Test progress; report the timer evidence either way
Noisy timings on a shared cluster	Repeats, median, record node names; compare only runs on the same node type
Runs exceed the 10-minute limit	One repetition per job; reduce steps, never grid size, in scaling runs. The current templates ask for 15 min and loop 5 reps; fix in task A6

Milestone	Planned content	Status
M1	Proposal; Python prototype	Done

Milestone	Planned content	Status
M2	Serial C version, tests, checksum; baseline E1 through Slurm	Code done; cluster run to do
M3	OpenMP on the cluster; MPI strips and blocks with two-phase halo, same checksum as serial	To do
M4	Hybrid MPI+OpenMP; scaling runs (E2–E4)	To do
M5	Sensitivity, profiling, overlap optimization (E5–E8); GPU kernel (E9)	To do
Final	Report, REPRODUCE.md, EuroHPC scale-up plan, slides	To do

7. Scale-up to EuroHPC

The natural next step is a Spain-wide forecast at 25 m resolution (about 8×10^8 cells, ≈ 8 GB per scenario) over several hundred weather scenarios. One scenario at 25 m fits in a single GPU's memory, so on a GPU system such as Leonardo Booster or MareNostrum 5 ACC the natural mapping is **one scenario per GPU**, with hundreds of independent scenarios spread across nodes. Multi-node domain decomposition becomes necessary at 10 m (≈ 5×10^9 cells) or when coupling to an atmospheric model; that is where our strips-vs-blocks results matter. The node-hour request will be derived from measured throughput: node-hours = scenarios × cells × steps / (MCUPS per node × 10^6 × 3600), with the step count for a fixed simulated time (§2). Operationally, ensemble output of this kind could feed EFFIS, the Copernicus European Forest Fire Information System, which already publishes fire danger forecasts for Europe.

Cluster usage: short jobs only (≤ 10 minutes), at most two CPU nodes (64 cores) and one GPU; nothing heavy on the login node.

8. Where the work happens

Code: on GitHub (`github.com/sanadhamadd/firefront-hpc`). Everyone works on a branch and opens a pull request; someone else reviews it before it is merged into `main` (see `CONTRIBUTING.md`). Code can be written and tested on a laptop with small grids. **Runs:** on the SciTech cluster. Log in with `ssh`, `git clone` (later `git pull`) the repo, load the modules, run `make`, then submit with `sbatch slurm/....sbatch`. Logs land in `results/raw/` on the cluster; copy the ones used in the report back into the repo and commit them. Nothing timed is ever run on the login node.

9. Division of tasks

Sanad has already built the serial base (phase 0). Work then goes in order: **Alicia** brings it onto the cluster and finishes the single-node part, **Sanad** adds MPI, and then **Alejandro, Rodrigo and Khadeja** work in parallel on three independent branches. Each hand-over is a tagged commit plus a short note in `docs/handover.md` (how to build and run, measured numbers, known issues). Nobody starts their stage until the previous tag passes the checksum test. Names in phase 3 are a proposal; swap among yourselves if you prefer.

Phase	Who	Tasks	Done when
0 done	**Sanad**	Serial C simulator, rule tables, RNG, checksum; tests; Makefile; Slurm templates; environment capture; OpenMP pragma (not yet run on the cluster); NumPy prototype.	In `main`; `make test` passes
1 → M2	**Alicia**	A1 Cluster check: `sinfo, scontrol show node, lscpu, module avail, srun --mpi=list, nvidia-smi -L`; fill `env/modules.txt` and write `env/cluster.md`; fix §5 if needed. A2 `--ignitions N` option (seeded points, default 1) plus a test. A3 OpenMP checksum test for 1, 2, 4, 8 threads in `tests/run_all.sh`. A4 `tools/parse_logs.py` and `tools/make_plots.py` (REPRODUCE.md already refers to them). A5 STREAM benchmark in `src/bench/` plus a Slurm script; roofline number. A6 Fix the Slurm templates (≤ 10 min, one repetition per job) and add one script per experiment. A7 Run on the cluster: E1 baseline and OpenMP thread scaling (first part of E2).	Tag `v0.2-cluster`; E1 + thread-scaling logs in `results/raw/`; numbers in handover note
2 → M3	**Sanad**	S1 Cartesian communicator, 1-D strips, `MPI_Sendrecv` halo. S2 2-D blocks with two-phase exchange and `MPI_Type_vector` columns. S3 Burnt-area `MPI_Allreduce` every K steps; per-rank halo/reduction timers in the RESULT line. S4 Same checksum as serial for several rank counts, including grids that do not divide evenly. S5 `make mpi` target; run E2 (MPI part) and E5.	Tag `v0.3-mpi`; checksum equal for all rank counts; E2 + E5 logs

Phase	Who	Tasks	Done when
3a → M4	**Alejandro**	Hybrid MPI+OpenMP runs on 1 and 2 nodes: E3, E4, E6. Produces the scaling figures for the report with `make_plots.py`.	Hybrid same checksum as serial; figures in `results/figures/`
3b → M5	**Rodrigo**	Overlap optimization (`MPI_Isend/Irecv`, interior/boundary split, `MPI_Test` variant, behind `--overlap`); perf / LIKWID profiling; run E7, E8; bottleneck analysis against the roofline bound.	Branch merged; before/after table; bottleneck evidence
3c → M5	**Khadeja**	CUDA kernel with data resident on the GPU, OpenACC version to compare; multicore fallback if the queue is blocked; run E9 and the output part of E10; nvidia-smi / Nsight. Stretch: real DEM and land cover (`make_landscape.py`).	Branch merged; GPU vs CPU table incl. transfers
4 Final	**Everyone**	Report: each person writes the sections for what they built (Alicia §1–2, methodology and final edit; Sanad HPC implementation; Alejandro results; Rodrigo bottleneck + optimization; Khadeja technology matrix). Alicia: `REPRODUCE.md`. Sanad: EuroHPC scale-up plan. Slides: one per person, in the same order as the phases.	All deliverables submitted; another member reproduces E1 + one scaling run from REPRODUCE.md

10. Shared conventions

Topic	Rule
Branches	One branch per feature (`omp-cluster, halo-exchange, overlap, cuda-port`), pull request, reviewed by someone else. Merge only when the checksum matches serial.
Results format	Every run prints one `RESULT key=value` line. New fields (nodes, t_halo, t_reduce, …) are added at the end so old logs still parse. `parse_logs.py` turns all logs into `results/tables/all.csv`.
Slurm scripts	Templates in `slurm/`, parameters via `sbatch --export=ALL,NX=…`; logs to `results/raw/_%j.out` with an environment snapshot next to them; record `sacct` for every job.
Cluster use	Jobs ≤ 10 min; never run the simulator on the login node; GPU jobs request 1 GPU only.
Reproducibility	Fixed seeds; module versions and compiler flags in `env/`; commit hash in every RESULT line.